

# GRANITE EMBEDDING MULTILINGUAL R2 MODELS

Granite Embedding Team, IBM Research<sup>\*†</sup>

## ABSTRACT

We introduce the multilingual Granite Embedding R2 models, a family of encoder-based embedding models for enterprise-scale dense retrieval across 200+ languages. Extending our English-focused R2 release, these models add enhanced support for 52 languages and programming code, a 32,768-token context window (a 64x expansion over R1), and state-of-the-art overall performance across multilingual and cross-lingual text search, code retrieval, long-document search, and reasoning retrieval datasets. The release consists of two bi-encoder models based on the ModernBERT architecture with an expanded multilingual vocabulary: a 311M-parameter full-size, and a 97M-parameter compact model built via model pruning and vocabulary selection that achieves the highest retrieval score of any open multilingual embedding model under 100M parameters. The full-size also supports Matryoshka Representation Learning for flexible embedding dimensionality. Both models are trained on enterprise-appropriate data with governance oversight, and released under the Apache 2.0 license at <https://huggingface.co/collections/ibm-granite>, designed to support responsible use and enable unrestricted research and enterprise adoption.

Average Performance of Multilingual Embedding Models on 5 Benchmarks

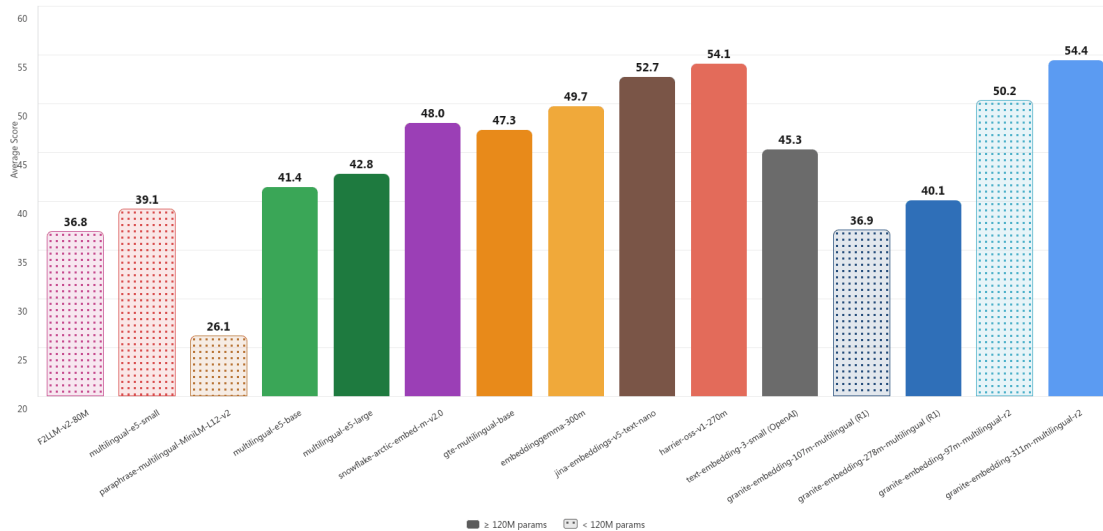


Figure 1: Performance of multilingual Granite R2 embedding models and comparable size open-source models, average multilingual MTEB performance. Refer section 4.1 for details.

<sup>\*</sup>See Section A for full author list. For questions, comments, compliments contact [awasthyps@us.ibm.com](mailto:awasthyps@us.ibm.com) or [raduf@us.ibm.com](mailto:raduf@us.ibm.com).

<sup>†</sup>For feedback or comments on this work, please open an issue at <https://github.com/ibm-granite/granite-embedding-models>.

## 1 INTRODUCTION

Bi-encoder text embedding models map text to a fixed-dimension vector such that semantically similar texts lie close in the vector space. These embeddings are most commonly used in retrieval, where document relevance to a query is scored by embedding similarity (Dunn et al., 2017; Xiong et al., 2020; Neelakantan et al., 2022; Zamani et al., 2018; Zhao et al., 2020), as well as in document clustering (Angelov, 2020) and text classification (Sun et al., 2019).

Encoder-based embedding models (Wang et al., 2022; Xiao et al., 2023; Chen et al., 2024; Merrick et al., 2024; Zhang et al., 2024; Wang et al., 2024a; Nussbaum et al., 2024; Awasthy et al., 2025b) are widely used for retrieval due to their low inference latency and small memory footprint compared to decoder-based models (Lee et al., 2024; Wang et al., 2023; Zhang et al., 2025; Akram et al., 2026). Building effective multilingual embedding models, however, poses additional challenges: the vocabulary must capture morphological diversity across many language families, the encoder must learn aligned representations across languages and scripts, and training data must cover a broad set of languages while maintaining quality. Many existing multilingual models are trained on data with non-commercial licenses, have limited context length, or provide uneven coverage across languages.

This report introduces the Granite Embedding Multilingual R2 models, purpose-built for multilingual information retrieval. These models provide substantial improvements over the R1 multilingual models (Awasthy et al., 2025a): a 64x expanded context length (from 512 to 32,768 tokens), support for 200+ languages with enhanced support for 52 languages and programming code, and an updated encoder based on the ModernBERT architecture (Warner et al., 2024) with a multilingual tokenizer trained on text and code across 200+ languages. Training data is curated and screened to remove personal information and profane language. Notably, we exclude the popular MS-MARCO dataset due to its non-commercial license. The models are released under the Apache 2.0 license. We provide two sizes to accommodate different inference budgets:

- `granite-embedding-311m-multilingual-r2` (311M parameters)<sup>1</sup>: the full-size model, with a 768-dimensional output embedding and Matryoshka Representation Learning support. Replaces `granite-embedding-278m-multilingual`.
- `granite-embedding-97m-multilingual-r2` (97M parameters)<sup>2</sup>: a compact model built via distillation and vocabulary selection, with a 384-dimensional output embedding. Replaces `granite-embedding-107m-multilingual`.

Both models deliver state-of-the-art overall performance across multilingual retrieval (MTEB-v2 Retrieval, BEIR), code retrieval (COIR), long-document search (LongEmbed), cross-lingual retrieval (MLQA), and reasoning retrieval (BRIGHT, RAR-b), while supporting 32,768-token contexts. The 311M full-size scores 65.2 on MTEB-v2 Retrieval Avg, placing it in the top 3 of open multilingual embedding models under 500M parameters. The 97M compact model scores 60.3 — the highest of any open multilingual embedding model under 100M parameters, with a 9+ point lead over the next-best model in its class.

The remainder of the paper is organized as follows: Section 2 describes training of the multilingual ModernBERT encoder. Section 3 details the bi-encoder training recipes, including the vocabulary selection approach for the compact model. Section 4 evaluates the Granite Embedding Multilingual models against other open-source multilingual encoder embedding models.

## 2 GRANITE MULTILINGUAL ENCODER MODELS

The Granite Embedding Multilingual R2 models feature updated encoder models, with longer context lengths, a richer multilingual training corpus, and modern architectural improvements. In this section, we discuss the architecture, training recipe and details of the high-quality multilingual corpus used to train the Granite Multilingual Encoder models. These models form the underlying backbone of the Granite Embedding models.

<sup>1</sup>[ibm-granite/granite-embedding-311m-multilingual-r2](#)

<sup>2</sup>[ibm-granite/granite-embedding-97m-multilingual-r2](#)

|                          | granite-encoder-small-multilingual | granite-encoder-multilingual |
|--------------------------|------------------------------------|------------------------------|
| Embedding size           | 384                                | 768                          |
| Layers                   | 12                                 | 22                           |
| Attention heads          | 12                                 | 12                           |
| Intermediate size        | 1536                               | 1152                         |
| Activation Function      | SiLU                               | GeLU                         |
| Vocabulary Size          | 180,000                            | 262,152                      |
| Max. Seq. Length Trained | 8,192                              | 8,192                        |
| Global Rope Theta        | 160,000                            | 160,000                      |
| Parameters               | ~97M                               | ~311M                        |
| Active Params            | ~28M                               | ~110M                        |

Table 1: Architectural Details for Granite Multilingual Encoder Models

## 2.1 ENCODER MODEL ARCHITECTURE

The Granite Multilingual Encoder models have been trained following the mmbert training recipe (Marone et al., 2025), including modern model optimizations introduced by ModernBERT (Warner et al., 2024) such as alternating attention mechanism, rotary positional embeddings for flexible context length, and streamlined parameters (eliminating unnecessary bias terms). The models also support Flash Attention (Dao, 2023) for improved efficiency. A key distinction from the English R2 encoder models is the use of an expanded multilingual tokenizer trained on code and text data across 200+ languages, with a vocabulary size of 262,152 for the base model and 180,000 for the compact model (via vocabulary selection). These encoder models have been trained on multilingual text and code data.

We train two models:

- *granite-encoder-multilingual* (311M parameters): the backbone of granite-embedding-311m-multilingual-r2. Its architecture follows ModernBERT-base, with 22 layers, a vector size of 768, and GeLU activation. Similar to ModernBERT, the model uses alternating global attention in every third layer.
- *granite-encoder-small-multilingual* (97M parameters): the backbone of granite-embedding-97m-multilingual-r2. Built via model pruning and vocabulary selection from the larger model, with 12 layers, a vector size of 384, SiLU activation, and a compact 180K-token vocabulary that preserves broad multilingual coverage while reducing model size by approximately 3x.

Detailed specifications of the architecture of each model are shown in Table 1.

The underlying encoder was pretrained on text from 200+ languages, and we report general-purpose embeddings for any of them. In addition, we provide enhanced support for 52 languages and programming code that receive explicit retrieval-pair and cross-lingual training data, producing higher-quality embeddings on retrieval tasks. The supported languages are listed in Appendix B.

## 2.2 TOKENIZER FERTILITY ANALYSIS

Tokenizer efficiency directly affects both throughput and effective context length: a model with a 32K-token window but twice the fertility of a competitor can effectively encode only half as much text. Tables 2 and 3 report tokenizer fertility (tokens per word) across natural languages and programming languages, respectively. Our full-size model uses the Gemma3 tokenizer (Team et al., 2025) with a 262K-token vocabulary (referred to as *granite-multi-262K*), while the compact variant uses a customized *gpt-oss* tokenizer pruned to 180K tokens (*granite-multi-180K*). On multilingual Wikipedia text, the Granite tokenizers exhibit 10% higher fertility than XLM-RoBERTa-based tokenizers (used by popular open source embedding models like *bge-m3* (Chen et al., 2024), *multilingual-e5* (Wang et al., 2024b), and *gte-multilingual* (Zhang et al., 2024)). However, on programming languages, both Granite tokenizers consistently achieve lower or comparable fertility relative to XLM-RoBERTa tokenizer, indicating more

efficient encoding of code data. We additionally include the unpruned `gpt-oss` tokenizer to show that pruning has a negligible effect on fertility.

| Tokenizer          | en   | ar   | de   | es   | hi   | ja   | ko   | ru   | sw   | te   | zh   | Avg. |
|--------------------|------|------|------|------|------|------|------|------|------|------|------|------|
| Granite-multi-262K | 1.60 | 2.12 | 2.00 | 1.56 | 1.52 | 0.68 | 0.79 | 2.24 | 2.12 | 3.02 | 0.77 | 1.67 |
| Granite-multi-180K | 1.50 | 1.96 | 1.92 | 1.52 | 1.81 | 0.82 | 0.81 | 2.25 | 1.89 | 3.24 | 0.86 | 1.69 |
| XLM-R-based, 250K  | 1.67 | 1.83 | 1.84 | 1.53 | 1.61 | 0.68 | 0.72 | 2.04 | 1.69 | 2.41 | 0.74 | 1.52 |
| GPT-OSS            | 1.50 | 1.96 | 1.92 | 1.52 | 1.81 | 0.82 | 0.81 | 2.25 | 1.89 | 3.24 | 0.86 | 1.69 |

Table 2: Tokenizer fertility (tokens per word) across natural languages. Lower values indicate more efficient encoding — fewer tokens per word allow more text to fit within a fixed context window. Measured on a sample of 10K sentences per language from Wikipedia.

| Tokenizer          | Python | Java | JavaScript | PHP  | Ruby | SQL  | C    | C++  | Avg. |
|--------------------|--------|------|------------|------|------|------|------|------|------|
| Granite-base-262K  | 3.46   | 3.04 | 3.82       | 3.41 | 3.52 | 3.49 | 3.18 | 3.38 | 3.41 |
| Granite-small-180K | 2.81   | 2.49 | 3.13       | 2.93 | 2.96 | 2.83 | 2.54 | 2.74 | 2.80 |
| XLM-R-based, 250K  | 3.73   | 3.32 | 4.16       | 3.78 | 3.55 | 3.58 | 3.30 | 3.62 | 3.63 |
| GPT-OSS            | 2.81   | 2.48 | 3.12       | 2.92 | 2.96 | 2.83 | 2.53 | 2.74 | 2.80 |

Table 3: Tokenizer fertility (tokens per word) across programming languages. Lower values indicate more efficient encoding — fewer tokens per word allow more text to fit within a fixed context window. Measured on a sample of 10K sentences per language from `github-code`, except for SQL which uses around 2.8K due to limited data availability.

### 2.3 TRAINING DATA

We curate a diverse, high-quality multilingual corpus of text and code to train our encoder models. The largest component of the training data is FineWeb2 (Penedo et al., 2025), from which we filter and retain data covering more than 1,800 languages throughout training. English-language data is primarily drawn from GneissWeb (Gohari et al., 2025), an IBM-curated dataset composed exclusively of open, commercial-friendly sources and rigorously filtered to produce high-quality corpora for language model training. For the last two stages of encoder training, which require higher-quality datasets, we use the filtered FineWeb-Edu (Lozhkov et al., 2024) for multilingual data. To enhance domain and stylistic diversity, we additionally include multilingual Wikipedia, Stack Exchange, and arXiv data. For improved performance on code-related tasks, we incorporate a subset of code data from the training corpora of the Granite Code models (Mishra et al., 2024).

We follow the language sampling strategy of mmBERT (Marone et al., 2025) to balance coverage between high-resource and low-resource languages throughout training. Specifically, we progressively increase the number of covered languages while decreasing the temperature used for language sampling from Stage 1 to Stage 3 (as described in 2.4), thereby promoting broader multilingual exposure in later stages. As a result, the final training stage covers more than 1,800 languages.

For tokenization, we use the Gemma3 tokenizer (Team et al., 2025) with a 262K-token vocabulary for the base model. The compact model employs a customized GPT-OSS tokenizer, further pruned to a 180K-token vocabulary, which preserves broad multilingual coverage while enabling a smaller model footprint.

### 2.4 TRAINING RECIPE

Following the ModernBERT training setting and mmBERT (Marone et al., 2025) recipe, we train our base encoder model on the Masked Language Modeling objective over three distinct stages:

1. Large Scale Pretraining: First, we train on 2.5 trillion tokens of multilingual text data, with a maximum context length of 1024. We use a Warmup-Stable-Decay learning rate schedule (Hu et al., 2024), with a RoPE theta of 10,000.

2. Context Extension: We then scale up the context length to 8192 and the RoPE theta to 160,000, continuing training on an additional 600 billion tokens at a constant learning rate.
3. Learning Rate Decay: Finally, we train with the same context length and RoPE theta as in Stage 2, but with a  $1-\sqrt{t}$  learning rate decay from the peak learning rate for 100 billion tokens. Following the mmBERT approach (Marone et al., 2025), we train three variants during this stage and linear merge them to form the final model:
  - (a) an English-focused variant with higher proportions of English data
  - (b) a multilingual-focused variant trained on data spanning more than 1,800 languages
  - (c) a variant continued directly from Stage 2

Across all stages, we use the StableAdamW optimizer (Wortsman et al., 2023), and employ efficient training mechanisms such as sequence packing, unpadding, and flash attention, as described in Warner et al. (2024).

The small model follows the same three-stage training recipe as the base model, with several adjustments. We initialize its weights from `granite-embedding-small-english-r2` (Awasthy et al., 2025b) by directly reusing the embedding rows for tokens shared with the source vocabulary and taking the average of all source embeddings for newly introduced tokens. This approach preserves learned representations for shared tokens while providing a reasonable initialization for new ones, enabling faster convergence during continued pretraining. In addition, during Stage 1, we reduce the masking rate and halve the learning rate and weight decay after 949 billion tokens. Finally, Stage 3 is extended to 200 billion tokens, compared to 100 billion tokens in the base model.

### 3 GRANITE EMBEDDING MULTILINGUAL R2

The Granite Embedding Multilingual R2 models are purpose-built for retrieval and trained on carefully curated, enterprise-ready data. This section describes the training data and methodology, covering contrastive finetuning, knowledge distillation, model merging, and pruning with vocabulary selection.

#### 3.1 TRAINING DATA

Granite Embedding Multilingual Models are trained on four types of data:

1. Unsupervised title-body paired data scraped from the web
2. Publicly available paired data with permissive, enterprise-friendly licenses
3. IBM product documentation paired data targeting specific technical domains
4. IBM-generated synthetic data, including multilingual long-document and short-passage data, and reasoning-oriented data

All data undergoes a clearance process with technical, business, and governance review, capturing content description, intended use, data classification, licensing, usage restrictions, and assessment of sensitive information (e.g., personal information).

The Multilingual R2 models reuse and extend the English Granite Embedding R2 training data (Awasthy et al., 2025a), adding multilingual and cross-lingual pairs:

- **Multilingual Retrieval Data:** retrieval pairs across the 52 enhanced-support languages, drawn from publicly available multilingual datasets and synthetically generated long- and short-passage pairs.
- **Cross-lingual Data:** pairs across multiple language combinations to enable cross-lingual retrieval (e.g., querying in one language and retrieving in another), including parallel translations of the same text.
- **Code Data:** code retrieval pairs from diverse sources, with hard negatives mined in most cases. Covers Python, Go, Java, JavaScript, PHP, Ruby, SQL, C++, and C.
- **Multi-Turn Conversational IR Data:** multi-turn data for conversational retrieval.

- **Synthetic Multilingual Data:** a synthetic data generation (SDG) pipeline producing passage-retrieval training data across 18 languages. Passages sampled from MIRACL corpora (where available) or Wikipedia serve as conditioning context for gpt-oss-120b, which generates question-answer pairs via few-shot prompting. Each query is augmented with 10 BM25 hard negatives, and data is produced at both passage and document granularity. We apply two rounds of LLM-as-judge filtering: removing false negatives from contrastive pairs, and scoring queries for human-likeness using language-specific prompts.
- **Synthetic Reasoning Data:** reasoning-oriented queries generated with gpt-oss-120b for documents from Arxiv, Pubmed, StackExchange Math, and StackOverflow, with hard negatives mined or generated following the method of Shao et al. (2025).

### 3.2 TRAINING RECIPE

Embedding models are typically trained with a contrastive objective (Gao et al., 2021) that pulls query embeddings toward relevant passages and pushes them away from non-relevant ones. The Granite Embedding Multilingual models are trained with the following pipeline:

1. **Contrastive Finetuning:** the models are first finetuned on a large corpus of multilingual paired data using the improved contrastive loss from Li et al. (2023). For a batch  $([q_i, (p_{ij})_j])_i$  of queries and passages — where  $p_{i0}$  is the positive passage for query  $i$  and  $p_{ij}, j > 0$  are negatives — the loss is:

$$\mathcal{L}_C = -\frac{1}{n} \sum_{i=1}^n \log \frac{e^{s(q_i, p_{i0})}}{Z_i}$$

$$Z_i = e^{s(q_i, p_{i0})} + \alpha \sum_{j>0} e^{s(q_i, p_{ij})} + \beta \sum_{i' \neq i} e^{s(q_i, q_{i'})} + \gamma \sum_{j>0} e^{s(p_{i0}, p_{ij})}$$

and  $s(q, p)$  is the temperature-scaled cosine similarity between the [CLS] embeddings of  $q$  and  $p$ :

$$s(q, p) = \frac{1}{\tau} \frac{\mathbf{E}(q)_{[\text{CLS}]} \cdot \mathbf{E}(p)_{[\text{CLS}]}}{\|\mathbf{E}(q)_{[\text{CLS}]}\| \|\mathbf{E}(p)_{[\text{CLS}]}\|}$$

We use a large batch size with in-batch negatives to better approximate the contrastive objective.

2. **Knowledge Distillation:** We distill from multiple teacher models into the student, minimizing the cross entropy between the teacher’s similarity-score distribution  $Sim_t$  and the student’s  $Sim_s$ . Following Hinton et al. (2014), both distributions are scaled by a temperature  $\tau_{KD}$ :

$$Sim_*(q_i, p_{ij}) = \frac{\exp(s_*(q_i, p_{ij})/\tau_{KD})}{\sum_{k=1}^m \exp(s_*(q_i, p_{ik})/\tau_{KD})}, \quad * \in \{t, s\},$$

and minimize

$$\mathcal{L}_{KD} = - \sum_{i=1}^n \sum_{j=1}^m Sim_t(q_i, p_{ij}) \log Sim_s(q_i, p_{ij}).$$

We use mined hard negatives and two teachers: one specialized on English retrieval, and one with stronger multilingual capabilities. For each homogeneous batch, the teacher is selected according to the language of the data.

3. **Context Extension:** Knowledge distillation is performed in two phases. The first uses a maximum sequence length of 512 with a large effective batch size; the second extends the maximum length to 4k and increases the RoPE theta to improve long-context performance.
4. **Model Merging:** To improve English performance, we train an identical model on English-only data and merge its weights with the multilingual model.
5. **Matryoshka Representation Learning (311M only):** The full-size model is trained with Matryoshka Representation Learning (Kusupati et al., 2024), allowing 768-dimensional embeddings to be truncated to 512, 384, 256, or 128 dimensions with graceful degradation, reducing storage, memory, and latency costs.

| Training Stage (cumulative) | ML MTEB-v2 Retrieval | MIRACL | MTEB Code |
|-----------------------------|----------------------|--------|-----------|
| Granite Encoder (base)      | 45.0                 | 43.5   | 36.4      |
| + Contrastive FT            | 55.8                 | 49.6   | 53.5      |
| + Knowledge Distillation    | 61.2                 | 60.0   | 62.0      |
| + Context Extension         | 63.7                 | 60.1   | 63.2      |
| + Model Merging             | 65.2                 | 59.8   | 63.9      |

Table 4: Cumulative effect of each training stage on granite-embedding-311m-multilingual-r2 model performance. Each row adds one stage to all previous stages, showing the marginal contribution of each step. We report the average NDCG@10 performance across all tasks of a benchmark.

### 3.3 TRAINING STAGE ABLATION

To validate the contribution of each pipeline stage, we evaluate the checkpoint after each stage on a representative set of benchmarks. Table 4 shows the cumulative effect of each stage on granite-embedding-311m-multilingual-r2. For the encoder, we report performance after finetuning for a few hundred steps on Miracl triples on the contrastive objective.

### 3.4 COMPACT MODEL: VOCABULARY SELECTION

The granite-embedding-97m-multilingual-r2 model uses a pruned multilingual tokenizer with 180K vocabulary tokens based on the GPT-OSS vocabulary, reduced from 200K by removing the most infrequent tokens, preserving broad coverage across 200+ languages at a smaller footprint.

The model is then finetuned with knowledge distillation and contrastive training. Despite being approximately 3x smaller than the full-size, it scores 60.3 on MTEB-v2 Retrieval Avg — the highest of any open multilingual embedding model under 100M parameters.

### 3.5 TEACHER TRAINING

We employ larger decoder models to be used as teachers, finetuning them on the contrastive learning objective to produce strong embeddings. Following the approach for teachers used in the English R2 models (Awasthy et al., 2025a), we merge high-performing checkpoints (selected with a held-out validation set), with either spherical interpolation or equal-weight linear interpolation using mergekit (Goddard et al., 2024), to create the final teacher models. After vigorous experiments with different models (from the Granite (Shah, 2026), Mistral (Jiang et al., 2023), Ministral (Liu et al., 2026) and Phi (Abdin et al., 2024) families), pooling strategies, hard negatives, instruction usage, and attention mechanism, we select two teachers for distilling the Granite Multilingual R2 embedding models:

- English Teacher: we train a Mistral 7B<sup>3</sup> teacher (Jiang et al., 2023) on only English data. This is the same teacher used in the English R2 models (Awasthy et al., 2025b).
- Multilingual Teachers: We use the multilingual data in Section 3.1 to train Granite 3.3 8B<sup>4</sup> and Granite 4.1 8B<sup>5</sup> teachers, finding the former teacher to be best for the 311M parameter embedding model and the latter to be better for the 97M embedding model.

For all teachers, we find last-token pooling using [EOS] to be an effective pooling mechanism for autoregressive decoders, as shown by others (Zhang et al., 2025; Akram et al., 2026). While the Mistral teacher is adapted to use bi-directional attention, we find that the Granite-based decoder teachers show strong performance out-of-the box with causal attention. We also find that using a separate teacher for English and Multilingual data helps improve overall performance, especially for the smaller model.

<sup>3</sup>mistralai/Mistral-7B-Instruct-v0.2

<sup>4</sup>ibm-granite/granite-3.3-8b-instruct

<sup>5</sup>ibm-granite/granite-4.1-8b

| Model                                         | Params<br>(M) | Embed.<br>Size | MTEB Retr.<br>Multi (18) | MTEB Code<br>(12) | MTEB Retr.<br>En (10) | LongEmbed<br>(6) | RaR-b<br>(17) |
|-----------------------------------------------|---------------|----------------|--------------------------|-------------------|-----------------------|------------------|---------------|
| f2llm-v2-80m                                  | 80            | 320            | 50.1                     | 68.0              | 47.5                  | 31.7             | 17.9          |
| multilingual-e5-small                         | 96            | 384            | 50.9                     | 51.3              | 46.5                  | 38.8             | 20.3          |
| multilingual-e5-base                          | 278           | 768            | 52.7                     | 52.6              | 49.0                  | 40.5             | 23.4          |
| snowflake-arctic-embed-m-v2.0                 | 305           | 768            | 54.8                     | 55.2              | 58.4                  | 55.4             | 23.3          |
| gte-multilingual-base                         | 305           | 768            | 57.2                     | 57.5              | 50.8                  | 62.1             | 19.0          |
| embeddinggemma-300m                           | 300           | 768            | 62.5                     | 69.0              | 54.6                  | 55.4             | 26.1          |
| jina-embeddings-v5-text-nano                  | 239           | 768            | 63.3                     | 71.2              | 58.8                  | 63.6             | 25.2          |
| harrier-oss-v1-270m                           | 270           | 640            | 66.4                     | 62.4              | 52.1                  | 65.0             | 32.9          |
| granite-embedding-107m-multilingual           | 107           | 384            | 48.1                     | 47.9              | 40.7                  | 34.3             | 17.1          |
| granite-embedding-278m-multilingual           | 278           | 768            | 52.2                     | 48.5              | 51.5                  | 37.7             | 18.9          |
| <b>granite-embedding-97m-multilingual-r2</b>  | <b>97</b>     | <b>384</b>     | <b>60.3</b>              | <b>60.5</b>       | <b>50.1</b>           | <b>62.9</b>      | <b>24.9</b>   |
| <b>granite-embedding-311m-multilingual-r2</b> | <b>311</b>    | <b>768</b>     | <b>65.2</b>              | <b>63.9</b>       | <b>52.6</b>           | <b>71.7</b>      | <b>28.0</b>   |

Table 5: Multilingual Retrieval Performance. Scores are averages across tasks, with the number of tasks indicated in parentheses. All scores are NDCG@10 unless otherwise noted.

## 4 EVALUATION

We evaluate the performance of our multilingual models on a variety of tasks and domains, spanning multilingual retrieval, code retrieval, long-document search, and reasoning retrieval.

### 4.1 RETRIEVAL PERFORMANCE

We evaluate the Granite Embedding Multilingual models on a variety of retrieval tasks, spanning multiple domains, languages, document lengths and text objects:

- **Multilingual Retrieval:** We evaluate on the MTEB-v2 Retrieval benchmark (Enevoldsen et al., 2025)
- **Code Retrieval:** We evaluate on code retrieval tasks of the MTEB-Code benchmark (Enevoldsen et al., 2025), including tasks from COIR (Li et al., 2024), which consists of text-to-code, code-to-text, and hybrid code retrieval.
- **English Retrieval:** We evaluate on general information retrieval benchmarks such as (Enevoldsen et al., 2025), comprising retrieval tasks on a variety of domains with a focus on zero-shot evaluations.
- **Long Context Retrieval:** To evaluate performance on retrieving long-context documents, we measure performance on the LongEmbed benchmark (Zhu et al., 2024).
- **Reasoning Retrieval:** To evaluate reasoning retrieval, we measure performance on the Reasoning-as-Retrieval benchmark (Xiao et al., 2024).

We compare our models with other state-of-the-art multilingual embedding models, including multilingual-e5-base (Wang et al., 2022), multilingual-e5-small, gte-multilingual-base (Zhang et al., 2024), snowflake-arctic-embed-m-v2.0 (Yu et al., 2024), embeddinggemma-300m (Vera et al., 2025), jina-embeddings-v5-text-nano (Akram et al., 2026), and harrier-oss-v1-270m. We also compare to the R1 Granite Embedding Multilingual Models, to quantify the improvement over the previous release.

#### 4.1.1 MULTILINGUAL RETRIEVAL PERFORMANCE

We evaluate multilingual performance of our models on a variety of open benchmarks, spanning across multiple domains. For competitor models, we take results from the MTEB leaderboard when available. For MTEB Retrieval (English and Multilingual), we use a maximum sequence length of 1024, for Code and RaR-b we use an MSL of 8192, and for LongEmbed we use an MSL of 32K (for models with shorter context lengths, we truncate to the model’s max seq. length.)

As shown in Table 5, the Granite Embedding Multilingual R2 models demonstrate strong performance across diverse multilingual tasks. The full-size granite-embedding-311m-multilingual-r2 scores 65.2 on



| Model                                         | Params<br>(M) | Embed.<br>Size | MTEB-v2<br>Retrieval (10) | MTEB Code<br>(12) | LongEmbed<br>(6) |
|-----------------------------------------------|---------------|----------------|---------------------------|-------------------|------------------|
| granite-embedding-small-english-r2            | 47            | 384            | 53.9                      | 55.8              | 63.7             |
| granite-embedding-english-r2                  | 149           | 768            | 56.4                      | 57.2              | 65.6             |
| <b>granite-embedding-97m-multilingual-r2</b>  | <b>97</b>     | <b>384</b>     | <b>50.1</b>               | <b>60.5</b>       | <b>65.5</b>      |
| <b>granite-embedding-311m-multilingual-r2</b> | <b>311</b>    | <b>768</b>     | <b>52.6</b>               | <b>63.9</b>       | <b>71.7</b>      |

Table 6: English-only retrieval performance comparison between the Granite English R2 and Multilingual R2 models, evaluated on the same English benchmarks. This quantifies the cost (if any) of multilingual capability on English tasks.

MTEB-v2 Retrieval Avg, a +13 point improvement over granite-embedding-278m-multilingual (52.2), placing it in the top 3 of open multilingual embedding models under 500M parameters on MTEB-v2 Retrieval Avg. The compact granite-embedding-97m-multilingual-r2, at just 97M parameters, scores 60.3 — a 9+ point lead over the next-best open model under 100M parameters.

#### 4.1.2 ENGLISH-ONLY PERFORMANCE: MULTILINGUAL VS. ENGLISH R2

To understand the tradeoff between multilingual coverage and English-specific performance, we evaluate our multilingual models on the same English-only benchmarks used for the English R2 models. Table 6 shows the comparison, with the English models performing better on English Retrieval, while the multilingual models showing stronger performance on Code and Long-document retrieval.

## 4.2 EMBEDDING SPEED

Text embedding models are fundamental to information retrieval systems and Retrieval-Augmented Generation (RAG) applications. Organizations typically process millions of documents, with frequent updates and new content requiring continuous ingestion. This makes encoding speed as important as accuracy—a slow model can become a significant bottleneck in large-scale deployments.

| Model                                  | Parameters<br>(M) ↓ | Embedding<br>Size | Encoding Speed<br>(Docs/s) ↑ | Rel to Granite<br>R2 equivalent |
|----------------------------------------|---------------------|-------------------|------------------------------|---------------------------------|
| F2LLM-v2-80M                           | 80                  | 320               | 2190                         | 86.4%                           |
| multilingual-e5-small                  | 96                  | 384               | 2604                         | 118.9%                          |
| granite-embedding-97m-multilingual-r2  | 97                  | 384               | 2534                         | 100.0%                          |
| granite-embedding-107m-multilingual    | 107                 | 384               | 3113                         | 122.9%                          |
| jina-embeddings-v5-text-nano           | 239                 | 768               | 307                          | 16.8%                           |
| harrier-oss-v1-270m                    | 270                 | 640               | 1938                         | 106.0%                          |
| multilingual-e5-base                   | 278                 | 768               | 2025                         | 110.8%                          |
| granite-embedding-278m-multilingual    | 278                 | 768               | 2164                         | 118.4%                          |
| embeddinggemma-300m                    | 300                 | 768               | 1349                         | 73.8%                           |
| gte-multilingual-base                  | 305                 | 768               | 2018                         | 110.4%                          |
| snowflake-arctic-embed-m-v2.0          | 305                 | 768               | 2190                         | 119.8%                          |
| granite-embedding-311m-multilingual-r2 | 311                 | 768               | 1828                         | 100.0%                          |

Table 7: Encoding speed comparison for multilingual models. All evaluations are done on a single Nvidia H100 GPU, with a batch size of 512 and truncating texts to 512 tokens max (to be comparable with 512 max models). The last column represents the relative speed of the given model to the size-equivalent granite multilingual R2 model. The evaluation was run with the version 5.8.0 of HuggingFace transformers - see Appendix C.5 for important speed considerations.

## 4.3 SPEED VS. ACCURACY

We compare the two R2 multilingual models against contemporary multilingual encoders of comparable scale. Quality is reported as the MTEB Multilingual Retrieval average across 18 languages; throughput

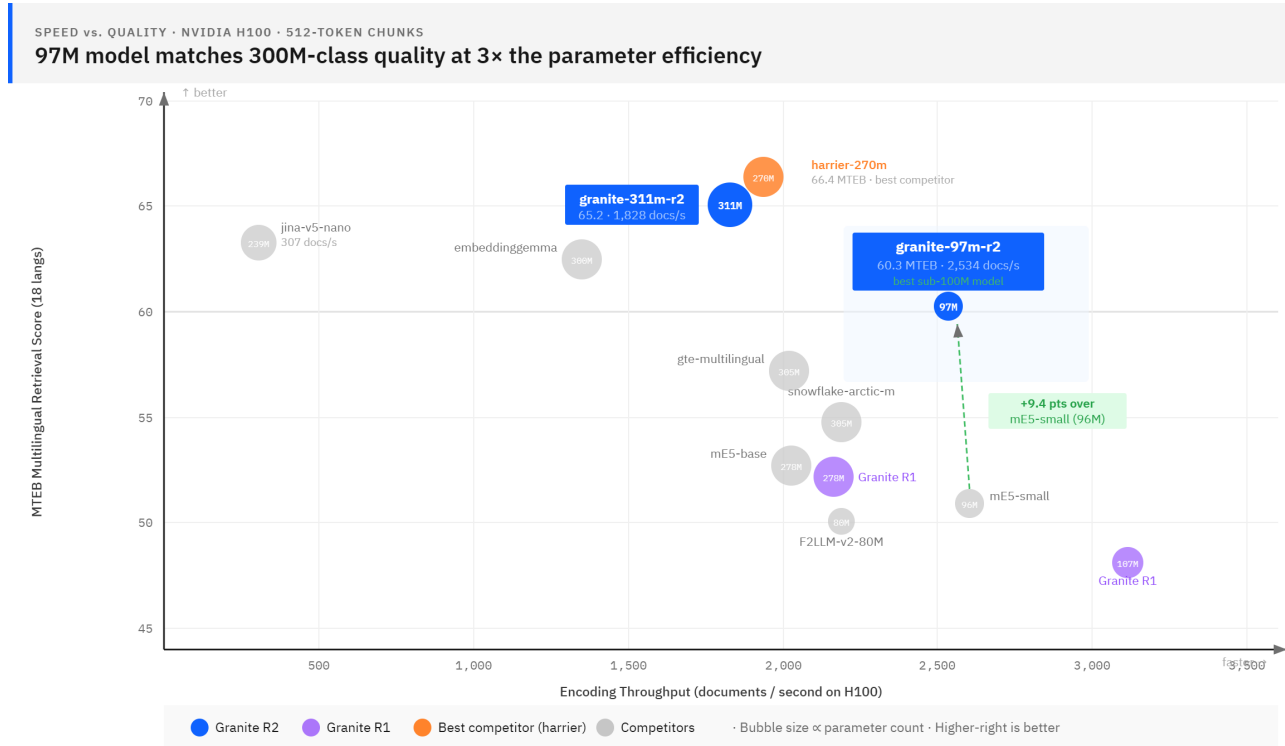


Figure 2: Speed vs. accuracy Pareto frontier for multilingual embedding models. The x-axis shows encoding speed (docs/sec on a single H100 GPU) and the y-axis shows MTEB-v2 Retrieval Avg. Models on or near the Pareto frontier offer the best tradeoff between throughput and retrieval quality. Granite multilingual R2 models are highlighted.

is measured in 512-token documents per second on a single NVIDIA H100 with batch size and sequence length held constant across models. Results are summarised in Figure 2 and Table 7.

At the 300M-parameter tier, granite-embedding-311m-multilingual-r2 reaches 65.2 MTEB at 1,828 docs/s. Among the surveyed models it is the second-strongest on retrieval quality, behind harrier-oss-v1-270m (66.4 MTEB, 1,938 docs/s), and ahead of embeddinggemma-300m, gte-multilingual-base, snowflake-arctic-embed-m-v2.0, and multilingual-e5-base. The R1 multilingual model at the same tier (278M parameters) scored substantially lower at comparable throughput, so the R2 release represents a large absolute gain in retrieval quality without a throughput regression in this size class.

At the sub-100M tier the gap to peer models is more pronounced. granite-embedding-97m-multilingual-r2 reaches 60.3 MTEB at 2,534 docs/s, exceeding multilingual-e5-small (96M, 50.9 MTEB at 2,604 docs/s) by 9.4 points at near-identical throughput, and improving on F2LLM-v2-80M by a similar margin. We are not aware of a publicly available model below 100M parameters that approaches this level of multilingual retrieval quality. Relative to its own larger sibling, the 97M model trails by 4.9 MTEB points while delivering  $\approx 1.4\times$  the throughput; relative to embeddinggemma-300m it is within a few points of quality at  $\approx 1.9\times$  the throughput and roughly one third of the parameter count.

The two R2 models therefore occupy distinct points on the speed–quality frontier: the 311M variant targets settings where retrieval quality dominates and a  $\sim 2,000$  docs/s encoding budget is acceptable, while the 97M variant targets latency- or memory-constrained settings where the prior best multilingual option in this class sacrificed close to ten points of MTEB quality. Neither model dominates on both axes against all peers — harrier-270m retains a small lead on quality at the 300M tier, and

`multilingual-e5-small` retains a small lead on raw throughput at the sub-100M tier — but both R2 models lie on the upper-right Pareto envelope of the configurations evaluated here.

## 5 CONCLUSION

In this work, we have presented the Granite Embedding Multilingual R2 Models, a family of specialized multilingual retrieval models designed to address the computational and accuracy requirements of enterprise-scale multilingual information retrieval systems. Our models support 200+ languages with enhanced support for 52 languages and programming code, a 32,768-token context window, and deliver state-of-the-art performance across diverse retrieval domains.

The proposed models incorporate several key contributions: (1) a full-size 311M-parameter multilingual embedding model that ranks in the top 3 of open multilingual models under 500M parameters on MTEB-v2 Retrieval, with Matryoshka Representation Learning for flexible embedding dimensionality; (2) a compact 97M-parameter multilingual model, built via model pruning and vocabulary selection, that supports long context and achieves the highest retrieval score of any open multilingual model under 100M parameters; and (3) comprehensive training on enterprise-appropriate data with transparent governance. We release these models under the Apache 2.0 license supporting both academic research and practical deployment scenarios.

## REFERENCES

- Marah Abdin, Jyoti Aneja, Harkirat Behl, Sébastien Bubeck, Ronen Eldan, Suriya Gunasekar, Michael Harrison, Russell J. Hewett, Mojan Javaheripi, Piero Kauffmann, James R. Lee, Yin Tat Lee, Yuanzhi Li, Weishung Liu, Caio C. T. Mendes, Anh Nguyen, Eric Price, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Xin Wang, Rachel Ward, Yue Wu, Dingli Yu, Cyril Zhang, and Yi Zhang. Phi-4 technical report, 2024. URL <https://arxiv.org/abs/2412.08905>.
- Mohammad Kalim Akram, Saba Sturua, Nastia Havriushenko, Quentin Herreros, Michael Günther, Maximilian Werk, and Han Xiao. jina-embeddings-v5-text: Task-targeted embedding distillation, 2026. URL <https://arxiv.org/abs/2602.15547>.
- Dimo Angelov. Top2vec: Distributed representations of topics. *CoRR*, abs/2008.09470, 2020. URL <https://arxiv.org/abs/2008.09470>.
- Parul Awasthy, Aashka Trivedi, Yulong Li, Mihaela Bornea, David Cox, Abraham Daniels, Martin Franz, Gabe Goodhart, Bhavani Iyer, Vishwajeet Kumar, Luis Lastras, Scott McCarley, Rudra Murthy, Vignesh P, Sara Rosenthal, Salim Roukos, Jaydeep Sen, Sukriti Sharma, Avirup Sil, Kate Soule, Arafat Sultan, and Radu Florian. Granite embedding models, 2025a. URL <https://arxiv.org/abs/2502.20204>.
- Parul Awasthy, Aashka Trivedi, Yulong Li, Meet Doshi, Riyaz Bhat, Vignesh P, Vishwajeet Kumar, Yushu Yang, Bhavani Iyer, Abraham Daniels, Rudra Murthy, Ken Barker, Martin Franz, Madison Lee, Todd Ward, Salim Roukos, David Cox, Luis Lastras, Jaydeep Sen, and Radu Florian. Granite embedding r2 models, 2025b. URL <https://arxiv.org/abs/2508.21085>.
- Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, 2024.
- Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023.
- Matthew Dunn, Levent Sagun, Mike Higgins, V. Ugur Guney, Volkan Cirik, and Kyunghyun Cho. Searchqa: A new q&a dataset augmented with context from a search engine, 2017.
- Kenneth Enevoldsen, Isaac Chung, Imene Kerboua, Márton Kardos, Ashwin Mathur, David Stap, Jay Gala, Wissam Siblini, Dominik Krzemiński, Genta Indra Winata, Saba Sturua, Saiteja Utpala, Mathieu Ciancone, Marion Schaeffer, Gabriel Sequeira, Diganta Misra, Shreeya Dhakal, Jonathan Rystrom, Roman Solomatin, Ömer Çağatan, Akash Kundu, Martin Bernstorff, Shitao Xiao, Akshita Sukhlecha, Bhavish Pahwa, Rafał Poświata, Kranthi Kiran GV, Shawon Ashraf, Daniel Auras, Björn Plüster,

- Jan Philipp Harries, Loïc Magne, Isabelle Mohr, Mariya Hendriksen, Dawei Zhu, Hippolyte Gisserot-Boukhlef, Tom Aarsen, Jan Kostkan, Konrad Wojtasik, Taemin Lee, Marek Šuppa, Crystina Zhang, Roberta Rocca, Mohammed Hamdy, Andrianos Michail, John Yang, Manuel Faysse, Aleksei Vatin, Nandan Thakur, Manan Dey, Dipam Vasani, Pranjal Chitale, Simone Tedeschi, Nguyen Tai, Artem Snegirev, Michael Günther, Mengzhou Xia, Weijia Shi, Xing Han Lù, Jordan Clive, Gayatri Krishnakumar, Anna Maksimova, Silvan Wehrli, Maria Tikhonova, Henil Panchal, Aleksandr Abramov, Malte Ostendorff, Zheng Liu, Simon Clematide, Lester James Miranda, Alena Fenogenova, Guangyu Song, Ruqiya Bin Safi, Wen-Ding Li, Alessia Borghini, Federico Cassano, Hongjin Su, Jimmy Lin, Howard Yen, Lasse Hansen, Sara Hooker, Chenghao Xiao, Vaibhav Adlakha, Orion Weller, Siva Reddy, and Niklas Muennighoff. Mmteb: Massive multilingual text embedding benchmark. *arXiv preprint arXiv:2502.13595*, 2025. doi: 10.48550/arXiv.2502.13595. URL <https://arxiv.org/abs/2502.13595>.
- Tianyu Gao, Xingcheng Yao, and Danqi Chen. SimCSE: Simple contrastive learning of sentence embeddings. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (eds.), *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 6894–6910, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.552. URL <https://aclanthology.org/2021.emnlp-main.552>.
- Charles Goddard, Shamane Siriwardhana, Malikeh Ehghaghi, Luke Meyers, Vladimir Karpukhin, Brian Benedict, Mark McQuade, and Jacob Solawetz. Arcee’s MergeKit: A toolkit for merging large language models. In Franck Dernoncourt, Daniel Preoțiuc-Pietro, and Anastasia Shimorina (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pp. 477–485, Miami, Florida, US, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-industry.36. URL <https://aclanthology.org/2024.emnlp-industry.36>.
- Hajar Emami Gohari, Swanand Ravindra Kadhe, Syed Yousaf Shah, Constantin Adam, Abdulhamid Adebayo, Praneet Adusumilli, Farhan Ahmed, Nathalie Baracaldo Angel, Santosh Subhashrao Borse, Yuan-Chi Chang, Xuan-Hong Dang, Nirmal Desai, Revital Eres, Ran Iwamoto, Alexei Karve, Yan Koyfman, Wei-Han Lee, Changchang Liu, Boris Lublinsky, Takuyo Ohko, Pablo Pesce, Maroun Touma, Shiqiang Wang, Shalisha Witherspoon, Herbert Woisetschlager, David Wood, Kun-Lung Wu, Issei Yoshida, Syed Zawad, Petros Zerfos, Yi Zhou, and Bishwaranjan Bhattacharjee. Gneissweb: Preparing high quality data for llms at scale, 2025.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the Knowledge in a Neural Network. In *NeurIPS Deep Learning Workshop*, 2014.
- Shengding Hu, Yuge Tu, Xu Han, Ganqu Cui, Chaoqun He, Weilin Zhao, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Xinrong Zhang, Zhen Leng Thai, Chongyi Wang, Yuan Yao, Chenyang Zhao, Jie Zhou, Jie Cai, Zhongwu Zhai, Ning Ding, Chao Jia, Guoyang Zeng, dahai li, Zhiyuan Liu, and Maosong Sun. MiniCPM: Unveiling the potential of small language models with scalable training strategies. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=3X2L2Tfr0f>.
- Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. Mistral 7b, 2023. URL <https://arxiv.org/abs/2310.06825>.
- Aditya Kusupati, Gantavya Bhatt, Aniket Rege, Matthew Wallingford, Aditya Sinha, Vivek Ramanujan, William Howard-Snyder, Kaifeng Chen, Sham Kakade, Prateek Jain, and Ali Farhadi. Matryoshka representation learning, 2024. URL <https://arxiv.org/abs/2205.13147>.
- Chankyu Lee, Rajarshi Roy, Mengyao Xu, Jonathan Raiman, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. Nv-embed: Improved techniques for training llms as generalist embedding models, 2024.
- Xiangyang Li, Kuicai Dong, Yi Quan Lee, Wei Xia, Yichun Yin, Hao Zhang, Yong Liu, Yasheng Wang, and Ruiming Tang. Coir: A comprehensive benchmark for code information retrieval models, 2024. URL <https://arxiv.org/abs/2407.02883>.

- Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. Towards general text embeddings with multi-stage contrastive learning, 2023. URL <https://arxiv.org/abs/2308.03281>.
- Alexander H. Liu, Kartik Khandelwal, Sandeep Subramanian, Victor Jouault, Abhinav Rastogi, Adrien Sadé, Alan Jeffares, Albert Jiang, Alexandre Cahill, Alexandre Gavaudan, Alexandre Sablayrolles, Amélie Héliou, Amos You, Andy Ehrenberg, Andy Lo, Anton Eliseev, Antonia Calvi, Avinash Sooriyarachchi, Baptiste Bout, Baptiste Rozière, Baudouin De Monicault, Clémence Lanfranchi, Corentin Barreau, Cyprien Courtot, Daniele Grattarola, Darius Dabert, Diego de las Casas, Elliot Chane-Sane, Faruk Ahmed, Gabrielle Berrada, Gaëtan Ecrepont, Gauthier Guinet, Georgii Novikov, Guillaume Kunsch, Guillaume Lample, Guillaume Martin, Gunshi Gupta, Jan Ludziejewski, Jason Rute, Joachim Studnia, Jonas Amar, Joséphine Delas, Josselin Somerville Roberts, Karmesh Yadav, Khyathi Chandu, Kush Jain, Laurence Aitchison, Laurent Fainsin, Léonard Blier, Lingxiao Zhao, Louis Martin, Lucile Saulnier, Luyu Gao, Maarten Buyt, Margaret Jennings, Marie Pellat, Mark Prins, Mathieu Poirée, Mathilde Guillaumin, Matthieu Dinot, Matthieu Futral, Maxime Darrin, Maximilian Augustin, Mia Chiquier, Michel Schimpf, Nathan Grinsztajn, Neha Gupta, Nikhil Raghuraman, Olivier Bousquet, Olivier Duchenne, Patricia Wang, Patrick von Platen, Paul Jacob, Paul Wambergue, Paula Kurylowicz, Pavankumar Reddy Muddireddy, Philomène Chagniot, Pierre Stock, Praveesh Agrawal, Quentin Torroba, Romain Sauvestre, Roman Soletskyi, Rupert Menneer, Sagar Vaze, Samuel Barry, Sanchit Gandhi, Siddhant Waghjale, Siddharth Gandhi, Soham Ghosh, Srijan Mishra, Sumukh Aithal, Szymon Antoniak, Teven Le Scao, Théo Cachet, Theo Simon Sorg, Thibaut Lavril, Thiziri Nait Saada, Thomas Chabal, Thomas Foubert, Thomas Robert, Thomas Wang, Tim Lawson, Tom Bewley, Tom Bewley, Tom Edwards, Umar Jamil, Umberto Tomasini, Valeriia Nemychnikova, Van Phung, Vincent Maladière, Virgile Richard, Wassim Bouaziz, Wen-Ding Li, William Marshall, Xinghui Li, Xinyu Yang, Yassine El Ouahidi, Yihan Wang, Yunhao Tang, and Zaccharie Ramzi. Minstral 3, 2026. URL <https://arxiv.org/abs/2601.08584>.
- Anton Lozhkov, Loubna Ben Allal, Leandro von Werra, and Thomas Wolf. Fineweb-edu: the finest collection of educational content, 2024. URL <https://huggingface.co/datasets/HuggingFaceFW/fineweb-edu>.
- Marc Marone, Orion Weller, William Fleshman, Eugene Yang, Dawn Lawrie, and Benjamin Van Durme. mmbert: A modern multilingual encoder with annealed language learning, 2025. URL <https://arxiv.org/abs/2509.06888>.
- Luke Merrick, Danmei Xu, Gaurav Nuti, and Daniel Campos. Arctic-embed: Scalable, efficient, and accurate text embedding models, 2024.
- Mayank Mishra, Matt Stallone, Gaoyuan Zhang, Yikang Shen, Aditya Prasad, Adriana Meza Soria, Michele Merler, Parameswaran Selvam, Saptha Surendran, Shivdeep Singh, Manish Sethi, Xuan-Hong Dang, Pengyuan Li, Kun-Lung Wu, Syed Zawad, Andrew Coleman, Matthew White, Mark Lewis, Raju Pavuluri, Yan Koyfman, Boris Lublinsky, Maximilien de Bayser, Ibrahim Abdelaziz, Kinjal Basu, Mayank Agarwal, Yi Zhou, Chris Johnson, Aanchal Goyal, Hima Patel, Yousaf Shah, Petros Zefos, Heiko Ludwig, Asim Munawar, Maxwell Crouse, Pavan Kapanipathi, Shweta Salaria, Bob Calio, Sophia Wen, Seetharami Seelam, Brian Belgodere, Carlos Fonseca, Amith Singhee, Nirmal Desai, David D. Cox, Ruchir Puri, and Rameswar Panda. Granite code models: A family of open foundation models for code intelligence, 2024.
- Arvind Neelakantan, Tao Xu, Raul Puri, Alec Radford, Jesse Michael Han, Jerry Tworek, Qiming Yuan, Nikolas Tezak, Jong Wook Kim, Chris Hallacy, Johannes Heidecke, Pranav Shyam, Boris Power, Tyna Eloundou Nekoul, Girish Sastry, Gretchen Krueger, David Schnurr, Felipe Petroski Such, Kenny Hsu, Madeleine Thompson, Tabarak Khan, Toki Sherbakov, Joanne Jang, Peter Welinder, and Lilian Weng. Text and code embeddings by contrastive pre-training, 2022. URL <https://arxiv.org/abs/2201.10005>.
- Zach Nussbaum, John X. Morris, Brandon Duderstadt, and Andriy Mulyar. Nomic embed: Training a reproducible long context text embedder, 2024.
- Guilherme Penedo, Hynek Kydlíček, Vinko Sabolčec, Bettina Messmer, Negar Foroutan, Amir Hossein Kargar, Colin Raffel, Martin Jaggi, Leandro Von Werra, and Thomas Wolf. Fineweb2: One pipeline to scale them all – adapting pre-training data processing to every language, 2025. URL <https://arxiv.org/abs/2506.20920>.

Yousaf Shah. Granite 4.1 llms: How they’re built, 2026. URL "<https://huggingface.co/blog/ibm-granite/granite-4-1>".

Rulin Shao, Rui Qiao, Varsha Kishore, Niklas Muennighoff, Xi Victoria Lin, Daniela Rus, Bryan Kian Hsiang Low, Sewon Min, Wen tau Yih, Pang Wei Koh, and Luke Zettlemoyer. Reasonir: Training retrievers for reasoning tasks, 2025. URL <https://arxiv.org/abs/2504.20595>.

Chi Sun, Xipeng Qiu, Yige Xu, and Xuanjing Huang. How to fine-tune bert for text classification? In *China national conference on Chinese computational linguistics*, pp. 194–206. Springer, 2019.

Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, Gaël Liu, Francesco Visin, Kathleen Kenealy, Lucas Beyer, Xiaohai Zhai, Anton Tsitsulin, Robert Busa-Fekete, Alex Feng, Noveen Sachdeva, Benjamin Coleman, Yi Gao, Basil Mustafa, Iain Barr, Emilio Parisotto, David Tian, Matan Eyal, Colin Cherry, Jan-Thorsten Peter, Danila Sinopalnikov, Surya Bhupatiraju, Rishabh Agarwal, Mehran Kazemi, Dan Malkin, Ravin Kumar, David Vilar, Idan Brusilovsky, Jiaming Luo, Andreas Steiner, Abe Friesen, Abhanshu Sharma, Abheesht Sharma, Adi Mayrav Gilady, Adrian Goedeckemeyer, Alaa Saade, Alex Feng, Alexander Kolesnikov, Alexei Bendeby, Alvin Abdagic, Amit Vadi, András György, André Susano Pinto, Anil Das, Ankur Bapna, Antoine Miech, Antoine Yang, Antonia Paterson, Ashish Shenoy, Ayan Chakrabarti, Bilal Piot, Bo Wu, Bobak Shahriari, Bryce Petrini, Charlie Chen, Charline Le Lan, Christopher A. Choquette-Choo, CJ Carey, Cormac Brick, Daniel Deutsch, Danielle Eisenbud, Dee Cattle, Derek Cheng, Dimitris Paparas, Divyashree Shivakumar Sreepathihalli, Doug Reid, Dustin Tran, Dustin Zelle, Eric Noland, Erwin Huizenga, Eugene Kharitonov, Frederick Liu, Gagik Amirkhanyan, Glenn Cameron, Hadi Hashemi, Hanna Klimczak-Plucińska, Harman Singh, Harsh Mehta, Harshal Tushar Lehri, Hussein Hazimeh, Ian Ballantyne, Idan Szpektor, Ivan Nardini, Jean Pouget-Abadie, Jetha Chan, Joe Stanton, John Wieting, Jonathan Lai, Jordi Orbay, Joseph Fernandez, Josh Newlan, Ju yeong Ji, Jyotinder Singh, Kat Black, Kathy Yu, Kevin Hui, Kiran Vodrahalli, Klaus Greff, Linhai Qiu, Marcella Valentine, Marina Coelho, Marvin Ritter, Matt Hoffman, Matthew Watson, Mayank Chaturvedi, Michael Moynihan, Min Ma, Nabila Babar, Natasha Noy, Nathan Byrd, Nick Roy, Nikola Momchev, Nilay Chauhan, Noveen Sachdeva, Oskar Bunyan, Pankil Botarda, Paul Caron, Paul Kishan Rubenstein, Phil Culliton, Philipp Schmid, Pier Giuseppe Sessa, Pingmei Xu, Piotr Stanczyk, Pouya Tafti, Rakesh Shivanna, Renjie Wu, Renke Pan, Reza Rokni, Rob Willoughby, Rohith Vallu, Ryan Mullins, Sammy Jerome, Sara Smoot, Sertan Girgin, Shariq Iqbal, Shashir Reddy, Shruti Sheth, Siim Põder, Sijal Bhatnagar, Sindhu Raghuram Panyam, Sivan Eiger, Susan Zhang, Tianqi Liu, Trevor Yacovone, Tyler Liechty, Uday Kalra, Utku Evci, Vedant Misra, Vincent Roseberry, Vlad Feinberg, Vlad Kolesnikov, Woohyun Han, Woosuk Kwon, Xi Chen, Yinlam Chow, Yuvein Zhu, Zichuan Wei, Zoltan Egyed, Victor Cotruta, Minh Giang, Phoebe Kirk, Anand Rao, Kat Black, Nabila Babar, Jessica Lo, Erica Moreira, Luiz Gustavo Martins, Omar Sanseviero, Lucas Gonzalez, Zach Gleicher, Tris Warkentin, Vahab Mirrokni, Evan Senter, Eli Collins, Joelle Barral, Zoubin Ghahramani, Raia Hadsell, Yossi Matias, D. Sculley, Slav Petrov, Noah Fiedel, Noam Shazeer, Oriol Vinyals, Jeff Dean, Demis Hassabis, Koray Kavukcuoglu, Clement Farabet, Elena Buchatskaya, Jean-Baptiste Alayrac, Rohan Anil, Dmitry, Lepikhin, Sebastian Borgeaud, Olivier Bachem, Armand Joulin, Alek Andreev, Cassidy Hardin, Robert Dadashi, and Léonard Hussenot. Gemma 3 technical report, 2025. URL <https://arxiv.org/abs/2503.19786>.

Henrique Schechter Vera, Sahil Dua, Biao Zhang, Daniel Salz, Ryan Mullins, Sindhu Raghuram Panyam, Sara Smoot, Iftexhar Naim, Joe Zou, Feiyang Chen, Daniel Cer, Alice Lisak, Min Choi, Lucas Gonzalez, Omar Sanseviero, Glenn Cameron, Ian Ballantyne, Kat Black, Kaifeng Chen, Weiye Wang, Zhe Li, Gus Martins, Jinhyuk Lee, Mark Sherwood, Juyeong Ji, Renjie Wu, Jingxiao Zheng, Jyotinder Singh, Abheesht Sharma, Divyashree Sreepathihalli, Aashi Jain, Adham Elarabawy, AJ Co, Andreas Dumanoglou, Babak Samari, Ben Hora, Brian Potetz, Dahun Kim, Enrique Alfonseca, Fedor Moiseev, Feng Han, Frank Palma Gomez, Gustavo Hernández Ábrego, Heseng Zhang, Hui Hui, Jay Han, Karan Gill, Ke Chen, Koert Chen, Madhuri Shanbhogue, Michael Boratko, Paul Suganthan, Sai Meher Karthik Duddu, Sandeep Mariserla, Setareh Ariafer, Shanfeng Zhang, Shijie Zhang, Simon Baumgartner, Sonam Goenka, Steve Qiu, Tanmaya Dabral, Trevor Walker, Vikram Rao, Waleed Khawaja, Wenlei Zhou, Xiaoqi Ren, Ye Xia, Yichang Chen, Yi-Ting Chen, Zhe Dong, Zhongli Ding, Francesco Visin, Gaël Liu, Jiageng Zhang, Kathleen Kenealy, Michelle Casbon, Ravin Kumar, Thomas Mesnard, Zach Gleicher, Cormac Brick, Olivier Lacombe, Adam Roberts, Qin

- Yin, Yunhsuan Sung, Raphael Hoffmann, Tris Warkentin, Armand Joulin, Tom Duerig, and Mojtaba Seyedhosseini. Embeddinggemma: Powerful and lightweight text representations, 2025. URL <https://arxiv.org/abs/2509.20354>.
- Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.
- Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Improving text embeddings with large language models, 2023.
- Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Multilingual e5 text embeddings: A technical report. *arXiv preprint arXiv:2402.05672*, 2024a.
- Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Multilingual e5 text embeddings: A technical report, 2024b.
- Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, Nathan Cooper, Griffin Adams, Jeremy Howard, and Iacopo Poli. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference, 2024.
- Mitchell Wortsman, Tim Dettmers, Luke Zettlemoyer, Ari Morcos, Ali Farhadi, and Ludwig Schmidt. Stable and low-precision training for large-scale vision-language models, 2023.
- Chenghao Xiao, G Thomas Hudson, and Noura Al Moubayed. Rar-b: Reasoning as retrieval benchmark. *arXiv preprint arXiv:2404.06347*, 2024.
- Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-pack: Packaged resources to advance general chinese embedding, 2023.
- Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul Bennett, Junaid Ahmed, and Arnold Overwijk. Approximate nearest neighbor negative contrastive learning for dense text retrieval, 2020. URL <https://arxiv.org/abs/2007.00808>.
- Puxuan Yu, Luke Merrick, Gaurav Nuti, and Daniel Campos. Arctic-embed 2.0: Multilingual retrieval without compromise, 2024.
- Hamed Zamani, Mostafa Dehghani, W. Bruce Croft, Erik Learned-Miller, and Jaap Kamps. From neural re-ranking to neural ranking: Learning a sparse representation for inverted indexing. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management, CIKM '18*, pp. 497–506, New York, NY, USA, 2018. Association for Computing Machinery. ISBN 9781450360142. doi: 10.1145/3269206.3271800. URL <https://doi.org/10.1145/3269206.3271800>.
- Xin Zhang, Yanzhao Zhang, Dingkun Long, Wen Xie, Ziqi Dai, Jialong Tang, Huan Lin, Baosong Yang, Pengjun Xie, Fei Huang, Meishan Zhang, Wenjie Li, and Min Zhang. mgte: Generalized long-context text representation and reranking models for multilingual text retrieval, 2024. URL <https://arxiv.org/abs/2407.19669>.
- Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*, 2025.
- Tiancheng Zhao, Xiaopeng Lu, and Kyusong Lee. Sparta: Efficient open-domain question answering via sparse transformer matching retrieval, 2020. URL <https://arxiv.org/abs/2009.13013>.
- Dawei Zhu, Liang Wang, Nan Yang, Yifan Song, Wenhao Wu, Furu Wei, and Sujian Li. Longembed: Extending embedding models for long context retrieval, 2024.

## A CONTRIBUTIONS

The Granite R2 embedding models were truly the outcome of a successful collaboration across geographies led by Radu Florian - with contributions from IBM Watson Research Lab (WRL) lab and India Research Lab (IRL). Parul Awasthy was the challenge lead on the project overall, calling from WRL, with Jaydeep Sen coordinating the work from IRL. We are very grateful for the wonderful and successful collaboration across continents - looking forward to even better models!

### ENCODER MODEL TRAINING

Parul Awasthy, Aashka Trivedi, Yushu Yang

### RETRIEVER TRAINING

Parul Awasthy, Ken Barker, Yulong Li, Aashka Trivedi, Yushu Yang

### DATA AND EVALUATION

Parul Awasthy, Ken Barker, Juergen Bross, Meet Doshi, Radu Florian, Martin Franz, Bhavani Iyer, Vishwajeet Kumar, Yulong Li, Vignesh P, Aashka Trivedi, Todd Ward

### PRODUCT MANAGEMENT

Abraham Daniels, Madison Lee

### TECHNICAL LEADERSHIP

Parul Awasthy, Radu Florian, Luis Lastras, Jaydeep Sen

## B SUPPORTED LANGUAGES

The 52 enhanced-support languages are: Albanian (sq), Arabic (ar), Azerbaijani (az), Bengali (bn), Bulgarian (bg), Catalan (ca), Chinese (zh), Croatian (hr), Czech (cs), Danish (da), Dutch (nl), English (en), Estonian (et), Finnish (fi), French (fr), Georgian (ka), German (de), Greek (el), Hebrew (he), Hindi (hi), Hungarian (hu), Icelandic (is), Indonesian (id), Italian (it), Japanese (ja), Kazakh (kk), Khmer (km), Korean (ko), Latvian (lv), Lithuanian (lt), Malay (ms), Marathi (mr), Norwegian (no), Persian (fa), Polish (pl), Portuguese (pt), Romanian (ro), Russian (ru), Serbian (sr), Slovak (sk), Slovenian (sl), Spanish (es), Swahili (sw), Swedish (sv), Tagalog (tl), Telugu (te), Thai (th), Turkish (tr), Ukrainian (uk), Urdu (ur), Uzbek (uz), and Vietnamese (vi). Additionally, the models are trained on programming code (Python, Go, Java, JavaScript, PHP, Ruby, SQL, C, C++) and support cross-lingual code retrieval.

## C DETAILED RETRIEVER PERFORMANCE EVALUATION

### C.1 MATRYOSHKA DIMENSION REDUCTION

Granite-embedding-311m-multilingual-r2 has been trained with Matryoshka dimensions (Kusupati et al., 2024), which support truncating the original 768 embedding dimensions to smaller vectors of sizes 512, 384, 256 and 128. Truncation allows for smaller memory consumption, with a minor decrease in performance, as shown in Table 8.

### C.2 CODE RETRIEVAL PERFORMANCE

We show per code retrieval task performance of the Granite Multilingual models in Table 9 evaluated at a maximum sequence length of 32K.



| Dimension  | MTEB Retrieval<br>En. (10) | MTEB Code<br>(12) | MTEB Retrieval<br>Multi. (18) |
|------------|----------------------------|-------------------|-------------------------------|
| 768 (full) | 52.6                       | 63.9              | 65.2                          |
| 512        | 52.5                       | 63.8              | 65.1                          |
| 384        | 52.1                       | 63.7              | 65.0                          |
| 256        | 51.6                       | 63.4              | 64.7                          |
| 128        | 50.4                       | 62.3              | 63.7                          |

Table 8: Performance of granite-embedding-311m-multilingual-r2 at different Matryoshka embedding dimensions.

| Model                                         | Avg. | Apps | COIR<br>CSN | CESN | CFB<br>MT | CFB<br>ST | CSN<br>CCR | CSN  | CT<br>Contest | CT<br>DL | CosQA | Stack<br>OverFlow QA | Syn.<br>Text2sql |
|-----------------------------------------------|------|------|-------------|------|-----------|-----------|------------|------|---------------|----------|-------|----------------------|------------------|
| granite-embedding-107m-multilingual           | 40.7 | 4.5  | 46.3        | 48.1 | 23.5      | 58.5      | 38.3       | 72.7 | 43.7          | 18.2     | 28.3  | 66.6                 | 39.3             |
| granite-embedding-278m-multilingual           | 48.5 | 6.2  | 52.9        | 51.3 | 31.4      | 67.7      | 43         | 76.4 | 60.8          | 32.1     | 33.4  | 78.5                 | 47.8             |
| <b>granite-embedding-97m-multilingual-r2</b>  | 60.4 | 50.2 | 75.4        | 53.3 | 55.2      | 75.2      | 51.9       | 85.3 | 76.5          | 34.1     | 34    | 82.2                 | 52.3             |
| <b>granite-embedding-311m-multilingual-r2</b> | 63.9 | 60.6 | 77.6        | 56.2 | 57        | 77.2      | 55.5       | 86.9 | 83.2          | 34.4     | 35.4  | 86.6                 | 55.7             |

Table 9: Code Retrieval Performance on the MTEB Code Benchmark. All scores are NDCG@10. *CSN*, *CESN*, *CFB*, *CT* is short for *CodeSearchNet*, *CodeEdistSearchNet*, *CodeFeedBack*, *CodeTrans*, respectively.

| Model                                         | MSL   | LongEmbed |        |         |       |      |       |      |  |
|-----------------------------------------------|-------|-----------|--------|---------|-------|------|-------|------|--|
|                                               |       | NQA       | Needle | PassKey | QMSum | SFD  | 2WmQA | Avg. |  |
| granite-embedding-107m-multilingual           | 512   | 22.9      | 21.5   | 28.3    | 21.1  | 58.5 | 53.7  | 34.3 |  |
| granite-embedding-278m-multilingual           | 512   | 23.8      | 25.8   | 38.5    | 21.8  | 60.6 | 55.8  | 37.7 |  |
| <b>granite-embedding-97m-multilingual-r2</b>  | 32768 | 46.0      | 53.3   | 82.8    | 41.2  | 90.1 | 79.4  | 65.5 |  |
| <b>granite-embedding-311m-multilingual-r2</b> | 32768 | 51.1      | 55.5   | 95.5    | 47.5  | 95.7 | 85.2  | 71.1 |  |

Table 10: Long Context Retrieval Performance on LongEmbed. MSL depicts the maximum sequence length of the embedding model. All scores are NDCG@10 except the Needle and Passkey subsets, which report Accuracy@1. *NQA*, *SFD*, *2WmQA* is short for *NarrativeQA*, *SummScreenFD*, *2WikiMultihopQA*, respectively.

### C.3 LONG-CONTEXT RETRIEVAL PERFORMANCE

Table 10 shows the strong performance of the Granite Embedding Multilingual R2 models on the long context benchmark LongEmbed, evaluated at a maximum sequence length of 32K.

### C.4 PER-LANGUAGE MIRACL RETRIEVAL

We show the per-language performance of the Granite Multilingual models in Table 11.

### C.5 RUNTIME SPEED FOR MODERNBERT MODELS

In late April 2026, we observed that our ModernBERT-based embedding models had roughly halved in throughput, with no corresponding change in our own training or inference code. Initial debugging explored several plausible hypotheses — attention backend fallback ordering (whether the model was silently dropping from FlashAttention to SDPA or eager attention), tokenizer maximum-length configuration affecting padding behaviour, and normalisation-related issues — each of which turned out to be a dead end. The symptoms pointed in multiple directions, which made the regression difficult to localise.

The root cause was a HuggingFace `transformers` library upgrade from version 4.57 to 5.1. In that upgrade, ModernBERT’s full-model unpadding optimization was removed as part of a broader effort to reduce technical debt and standardize model implementations across the library. The optimization

| Language        | granite-embeddings<br>-107m-multilingual | granite-embeddings<br>-278m-multilingual | granite-embeddings<br>-97m-multilingual-r2 | granite-embeddings<br>-311m-multilingual-r2 |
|-----------------|------------------------------------------|------------------------------------------|--------------------------------------------|---------------------------------------------|
| Arabic (ar)     | 63.98                                    | 65.74                                    | 63.76                                      | 67.93                                       |
| Bengali (bn)    | 65.48                                    | 68.05                                    | 66.29                                      | 70.85                                       |
| Chinese (zh)    | 52.02                                    | 54.91                                    | 55.07                                      | 58.35                                       |
| English (en)    | 49.04                                    | 51.13                                    | 45.39                                      | 45.46                                       |
| Finnish (fi)    | 66.19                                    | 67.92                                    | 65.62                                      | 68.47                                       |
| French (fr)     | 50.45                                    | 51.88                                    | 48.72                                      | 52.08                                       |
| German (de)     | 47.41                                    | 49.70                                    | 49.59                                      | 50.97                                       |
| Hindi (hi)      | 44.28                                    | 47.72                                    | 47.09                                      | 51.77                                       |
| Indonesian (id) | 46.04                                    | 47.51                                    | 45.98                                      | 46.93                                       |
| Japanese (ja)   | 60.87                                    | 63.51                                    | 58.37                                      | 62.52                                       |
| Korean (ko)     | 60.70                                    | 60.93                                    | 55.21                                      | 59.15                                       |
| Persian (fa)    | 48.76                                    | 51.83                                    | 49.09                                      | 52.39                                       |
| Russian (ru)    | 51.43                                    | 54.51                                    | 51.30                                      | 56.86                                       |
| Spanish (es)    | 49.71                                    | 51.94                                    | 48.64                                      | 51.01                                       |
| Swahili (sw)    | 59.11                                    | 61.27                                    | 57.70                                      | 66.18                                       |
| Telugu (te)     | 78.29                                    | 79.28                                    | 75.34                                      | 80.38                                       |
| Thai (th)       | 72.03                                    | 73.54                                    | 65.21                                      | 71.73                                       |
| Yoruba (yo)     | 64.69                                    | 68.70                                    | 61.19                                      | 63.57                                       |
| <b>Average</b>  | 57.25                                    | 59.45                                    | 56.09                                      | 59.81                                       |

Table 11: Per-Language MIRACL Retrieval Performance (nDCG@10).

had allowed the model to skip computation on padding tokens throughout the entire transformer stack, which provided a substantial speedup for batches with variable-length sequences. The removal was a reasonable architectural decision — maintaining bespoke optimization paths for individual models creates long-term maintenance burden — but the practical effect for downstream users is that the model now processes padding tokens through every layer, effectively doubling compute for typical mixed-length workloads. Table 12 reports throughput under the pinned earlier release (4.57.6), where the optimization is still present.

In Table 13 we show the impact of changing the code from release 4.57.6 to 5.8.0, to make the reader aware of the change. Note that there is a workaround, using a different collator, that not only recovers the performance drop, but actually improves performance relative to 4.57.6, but you need (at this point) custom code in the SentenceTransformer.encode method for it. For details, see our jupyter notebook here [Granite Embedding Collator notebook](#).

This case illustrates a recurring challenge at the boundary between ML infrastructure and upstream libraries: performance characteristics can depend on internal implementation details that may legitimately change as libraries evolve. A routine version upgrade can shift throughput without any functional change in model outputs, and diagnosing the cause requires bisecting across dependency versions rather than commits in the model repository — a search space that is easy to overlook when the natural first instinct is to inspect one’s own code.

## D CONTEXT LENGTH SCALING ANALYSIS

To demonstrate that the extended 32,768-token context window provides practical benefits, we evaluate retrieval performance as a function of maximum sequence length at inference time. Table 14 shows performance on the LongEmbed benchmark improves as the allowed context length increases.

| Model                                         | Params<br>(M) ↓ | Emb.<br>Size | Infer. Speed<br>(spans/s) ↑ | Rel to Granite<br>R2 equivalent |
|-----------------------------------------------|-----------------|--------------|-----------------------------|---------------------------------|
| F2LLM-v2-80M                                  | 80              | 320          | 2645                        | 80.9%                           |
| multilingual-e5-small                         | 96              | 384          | 2670                        | 81.7%                           |
| <b>granite-embedding-97m-multilingual-r2</b>  | <b>97</b>       | <b>384</b>   | <b>3268</b>                 | <b>100.0%</b>                   |
| granite-embedding-107m-multilingual           | 107             | 384          | 2941                        | 90.0%                           |
| jina-embeddings-v5-text-nano                  | 239             | 768          | 1115                        | 37.7%                           |
| harrier-oss-v1-270m                           | 270             | 640          | 2004                        | 67.7%                           |
| multilingual-e5-base                          | 278             | 768          | 2084                        | 70.4%                           |
| granite-embedding-278m-multilingual           | 287             | 768          | 2117                        | 71.5%                           |
| embeddinggemma-300m                           | 300             | 768          | 1256                        | 42.4%                           |
| gte-multilingual-base                         | 305             | 768          | —                           | —                               |
| snowflake-arctic-embed-m-v2.0                 | 305             | 768          | —                           | —                               |
| <b>granite-embedding-311m-multilingual-r2</b> | <b>311</b>      | <b>768</b>   | <b>2960</b>                 | <b>100.0%</b>                   |

Table 12: Encoding throughput (documents per second) on a single NVIDIA H100 with 512-token inputs, measured with Hugging Face `transformers` 4.57.6. We observed a throughput regression in version 5.1.0 affecting all evaluated models, so we report numbers under the pinned earlier release. Relative speed compares each model to its tier’s Granite R2 reference: granite-97m-r2 for sub-110M models and granite-311m-r2 for 230M+ models. Dashes mark models not yet re-measured under the pinned toolkit version.

| Model                                         | Params<br>(M) | Embed.<br>Size | 4.57.6<br>(docs/s) | 5.8.0<br>(docs/s) | $\Delta$      |
|-----------------------------------------------|---------------|----------------|--------------------|-------------------|---------------|
| F2LLM-v2-80M                                  | 80            | 320            | 2,645              | 2,190             | −17.2%        |
| multilingual-e5-small                         | 96            | 384            | 2,670              | 2,604             | −2.5%         |
| <b>granite-embedding-97m-multilingual-r2</b>  | <b>97</b>     | <b>384</b>     | <b>3,268</b>       | <b>2,534</b>      | <b>−22.5%</b> |
| granite-embedding-107m-multilingual           | 107           | 384            | 2,941              | 3,113             | +5.8%         |
| jina-embeddings-v5-text-nano                  | 239           | 768            | 1,115              | 307               | −72.5%        |
| harrier-oss-v1-270m                           | 270           | 640            | 2,004              | 1,938             | −3.3%         |
| multilingual-e5-base                          | 278           | 768            | 2,084              | 2,025             | −2.8%         |
| granite-embedding-278m-multilingual           | 287           | 768            | 2,117              | 2,164             | +2.2%         |
| embeddinggemma-300m                           | 300           | 768            | 1,256              | 1,349             | +7.4%         |
| gte-multilingual-base                         | 305           | 768            | —                  | 2,018             | —             |
| snowflake-arctic-embed-m-v2.0                 | 305           | 768            | —                  | 2,190             | —             |
| <b>granite-embedding-311m-multilingual-r2</b> | <b>311</b>    | <b>768</b>     | <b>2,960</b>       | <b>1,828</b>      | <b>−38.2%</b> |

Table 13: Encoding throughput (documents per second) on a single NVIDIA H100 with batch size 512 and 512-token inputs, measured under two HuggingFace `transformers` releases: 4.57.6 (with full-model unpadding for ModernBERT) and 5.8.0 (after the optimization was removed).  $\Delta$  reports the relative change from 4.57.6 to 5.8.0. The regression is concentrated in ModernBERT-based models (Granite R2, jina-v5); XLM-RoBERTa-based competitors are within a few percent across the two releases. Dashes mark configurations not yet re-measured on the pinned earlier release.

| <i>Max Seq. Len</i> →                  | 512  | 1024 | 2048 | 4096 | 8192 | 16384 | 32768 |
|----------------------------------------|------|------|------|------|------|-------|-------|
| gte-multilingual-base (max 8192)       | 38.2 | 43.6 | 49.7 | 59.1 | 62.1 | —     | —     |
| granite-embedding-97m-multilingual-r2  | 39.1 | 46.4 | 52.9 | 59.0 | 62.9 | 64.3  | 65.5  |
| granite-embedding-311m-multilingual-r2 | 40.3 | 47.7 | 54.1 | 61.3 | 67.1 | 70.7  | 71.7  |

Table 14: Effect of maximum sequence length at inference on long-context retrieval performance on LongEmbed Benchmark. Models are evaluated with truncation at varying sequence lengths to show how performance scales with available context. Competitors are capped at their native max sequence length. Average scores reported across the 6 tasks of the LongEmbed benchmark

## E RETRIEVER TRAINING HYPERPARAMETERS

The hyperparameters used for different stages of retrieval training are indicated in Table 15.

| Model                                  | Stage                  | LR     | Batch Size | Steps | Seq. Len | Rope Theta |
|----------------------------------------|------------------------|--------|------------|-------|----------|------------|
| granite-embedding-311m-multilingual-r2 | Contrastive FT         | $1e-4$ | 9600       | 40000 | 512      | 10000      |
|                                        | Contrastive KD         | $8e-5$ | 128        | 8000  | 512      | 10000      |
|                                        | Long-Context Extension | $8e-6$ | 32         | 10000 | 4096     | 150000     |
| granite-embedding-97m-multilingual-r2  | Contrastive FT         | $1e-4$ | 9600       | 40000 | 512      | 10000      |
|                                        | Contrastive KD         | $8e-5$ | 256        | 12000 | 512      | 10000      |
|                                        | Long-Context Extension | $8e-6$ | 128        | 10000 | 4096     | 150000     |

Table 15: Retriever Training Hyperparameters. Batch size refers to the global batch size, and rope theta refers to the global rope theta. FT and KD refers to finetuning and knowledge distillation respectively.